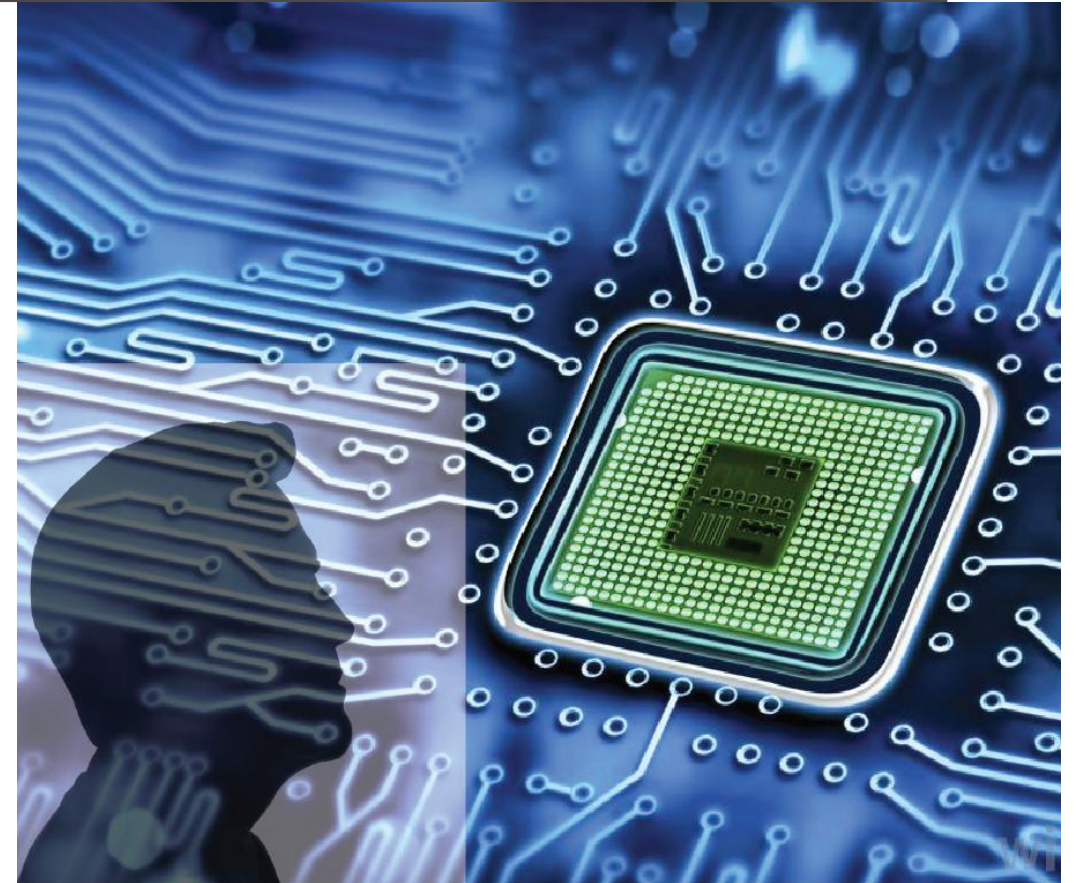


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-3, Lecture-1 SystemC

Sneh Saurabh
26th August, 2025



SystemC

Immediate and Delayed Notification: Example

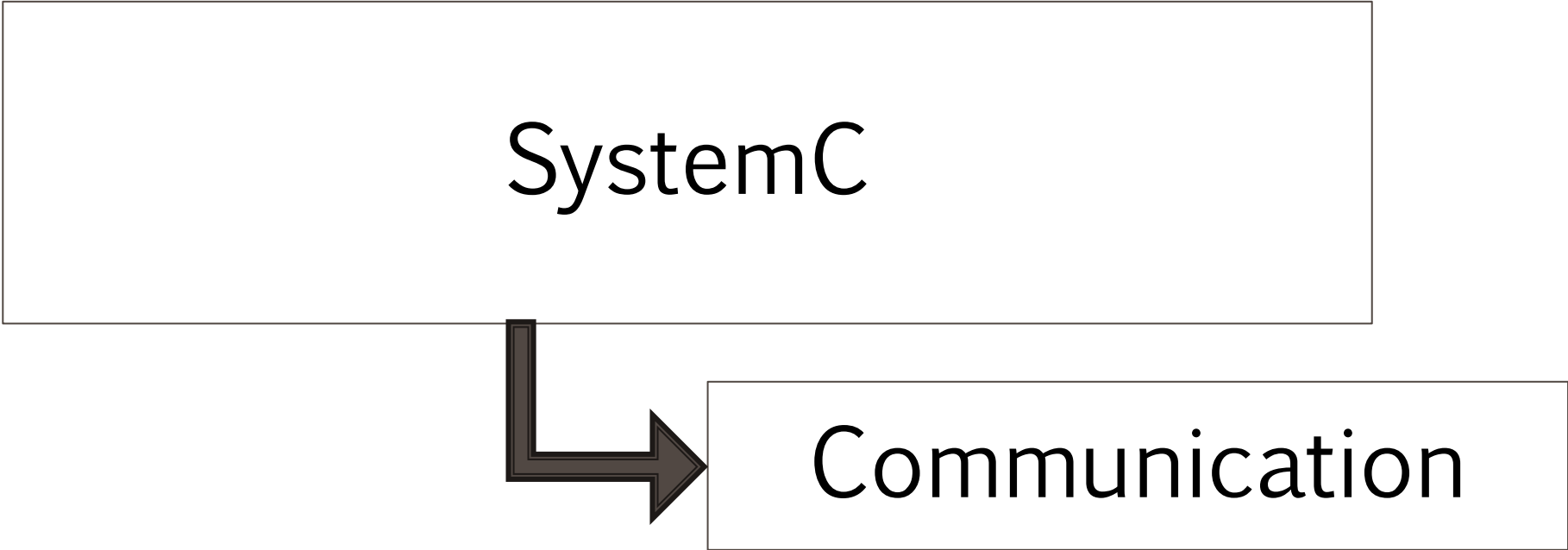
```
SC_MODULE(signal_processes) {  
    void sender(void); // process  
    void receiver_immediate(void); // process  
    void receiver_delayed(void); // process  
    sc_signal<bool> sig;  
    sc_event immediate;  
    sc_event delayed;  
    SC_CTOR(signal_processes) {  
        SC_THREAD(sender); // register process  
        SC_THREAD(receiver_immediate); // register process  
        SC_THREAD(receiver_delayed); // register process  
        sig.write(false);  
    }  
};
```

- Two threads wait on events
- `receiver_immediate()` gets triggered by immediate notification
- `receiver_delayed()` gets triggered by delayed notification

```
Initial:  sig=0 T=0 s  
Immediate: sig=0 T=0 s  
Delayed:  sig=1 T=0 s
```

```
void signal_processes::sender(void) {  
    wait(SC_ZERO_TIME);  
    cout << "Initial: " << " sig=" <<  
        sig.read() << " T=" << sc_time_stamp() << endl;  
    sig.write(true);  
    delayed.notify(SC_ZERO_TIME);  
    immediate.notify();  
}  
void signal_processes::receiver_immediate(void) {  
    wait(immediate);  
    cout << "Immediate: " << " sig=" <<  
        sig.read() << " T=" << sc_time_stamp() << endl;  
}  
void signal_processes::receiver_delayed(void) {  
    wait(delayed);  
    cout << "Delayed: " << " sig=" <<  
        sig.read() << " T=" << sc_time_stamp() << endl;  
}
```

- A process gets triggered by an event only when it is waiting for that event
 - To allow both receiver processes to be waiting before the event `wait(SC_ZERO_TIME)` is added in `sender()`



```
graph TD; A[SystemC] --> B[Communication]
```

SystemC

Communication

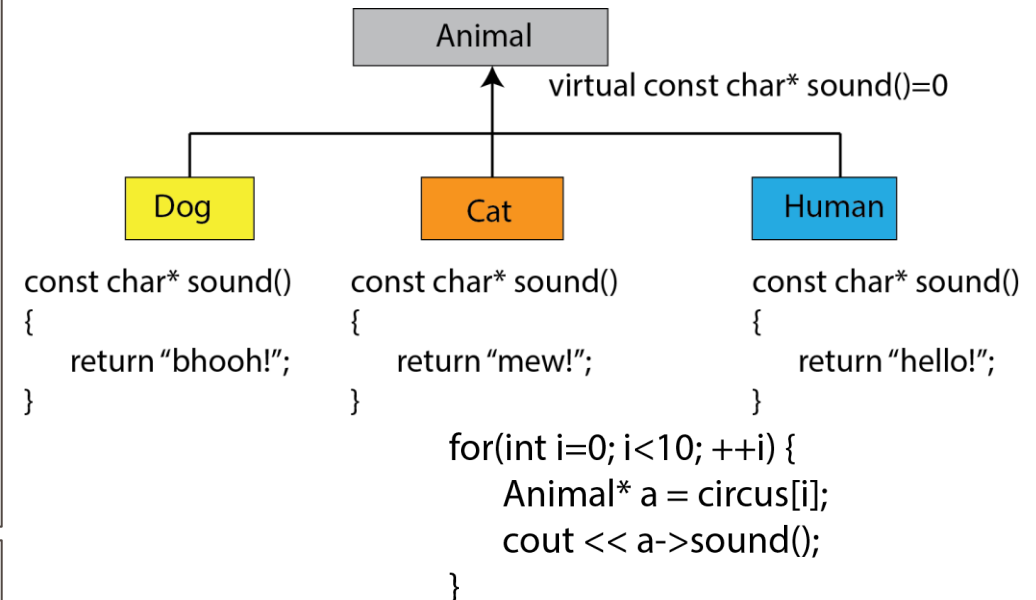
Interfaces, Channels and Ports (1)

Abstract Classes in C++

- Abstract classes can be created by declaring at least one **pure virtual member function**
- Cannot be instantiated, but their pointer and references can be used
- Provides expressions of general concepts from which more specific classes can be derived

Interfaces

- A SystemC interface is an **abstract class** that inherits from **sc_interface** and provides only **pure virtual declarations** of methods referenced by SystemC channels and ports.
- No implementations or data are provided in a SystemC interface.

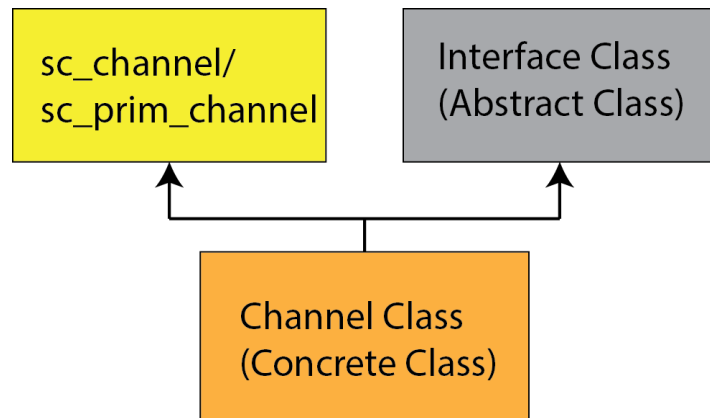


```
class my_interface: public sc_interface
{
    public:
        virtual int read(void) = 0;
        virtual void write(int i) = 0;
};
```

Interfaces, Channels and Ports (2)

Channels

- A SystemC channel is a class that inherits from either `sc_channel` or from `sc_prim_channel`, and the channel should inherit and implement one or more **SystemC interface classes**.
- A channel implements all the pure virtual methods of the inherited interface classes.



- By using interfaces to connect channels, we can implement modules independent of the communication

```
class my_interface: public sc_interface
{
    public:
        virtual int read(void) = 0;
        virtual void write(int i) = 0;
};

class my_channel: public sc_prim_channel, public my_interface
{
    private:
        int _data;
    public:
        int read(void) { return _data; }
        void write(int i) { _data = i; }
};
```

Interfaces, Channels and Ports (3)

Ports

- A SystemC port is created using a class in which an interface class is its template argument (inherits its member functions)

`sc_port<interface> portName;`

- A port is declared as a data member inside a module

- Port provides the means by which a module can be written such that it is independent of the context in which it is instantiated

```
SC_MODULE(computation) {  
    void compute(void); // process  
    sc_port<my_interface> marks_po;  
    sc_port<my_interface> grade_po;  
    SC_CTOR(computation) {  
        SC_THREAD(compute); // register process  
    }  
};
```

Interfaces, Channels and Ports (4)

How to connect a port?

- Instantiate the module
- Instantiate the appropriate channel
- Connect port with channel

```
my_channel mc_marks;  
my_channel mc_grade;  
computation computation_i("computation_i");  
computation_i.marks_po(mc_marks); // bind port to channel  
computation_i.grade_po(mc_grade); // bind port to channel
```

- The channel object is taken by reference and its pointer/reference is saved for later access by the port
 - Allow access to channels (and its member functions) across module boundaries

Summary:

- Port requires services
 - Interface defines services
 - Channel implements services
-
- Port can bind to a channel if the channel implements the services needed by the port
 - In effect: the port is an interface **pointer to a channel** that implements the interface

Interfaces, Channels and Ports: Example

```
class my_interface: public sc_interface
{
    public:
        virtual int read(void) = 0;
        virtual void write(int i) = 0;
};

class my_channel: public sc_prim_channel, public my_interface
{
    private:
        int _data;
    public:
        int read(void) { return _data; }
        void write(int i) { _data = i; }
};
```

```
SC_MODULE(computation) {
    void compute(void); // process
    sc_port<my_interface> marks_po;
    sc_port<my_interface> grade_po;
    SC_CTOR(computation) {
        SC_THREAD(compute); // register process
    }
};
```

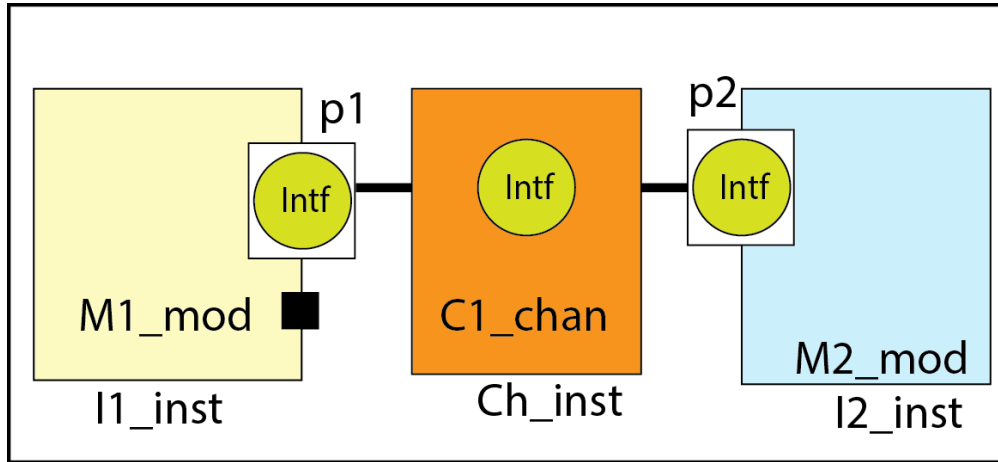
```
void computation::compute(void) {
    int mk = marks_po->read(); // read from port
    int gr = 2;
    if (mk > 80) gr = 10;
    else if (mk > 60) gr = 7;
    else if (mk > 40) gr = 5;
    grade_po->write(gr); // write to the port
}
```

```
int sc_main(int sc_argc, char* sc_argv[]) {
    //create an instance of the SystemC module
    my_channel mc_marks;
    my_channel mc_grade;
    computation computation_i("computation_i");
    computation_i.marks_po(mc_marks); // bind port to channel
    computation_i.grade_po(mc_grade); // bind port to channel
    mc_marks.write(75); // write to the channel which automatically writes to port
    cout << "Marks from channel=" << mc_marks.read() << endl;
    cout << "Marks from module port=" << computation_i.marks_po->read() << endl;
    sc_start(1, SC_SEC); // invoke the simulator, computes
    cout << "Grade from channel=" << mc_grade.read() << endl;
    cout << "Grade from module port=" << computation_i.grade_po->read() << endl;
    return 0;
}
```

- Computation is done in a module (independent of the channel and using only interface functions)
 - Port acts like a pointer
- Module receives data from the channel, performs the computation in the process, and results are available in the channel

```
Marks from channel=75
Marks from module port=75
Grade from channel=7
Grade from module port=7
```

Instance Connections: sc_port



- Channel Instance outside the module connects two ports
- Ports of instances connected using channel
- Channel exposed

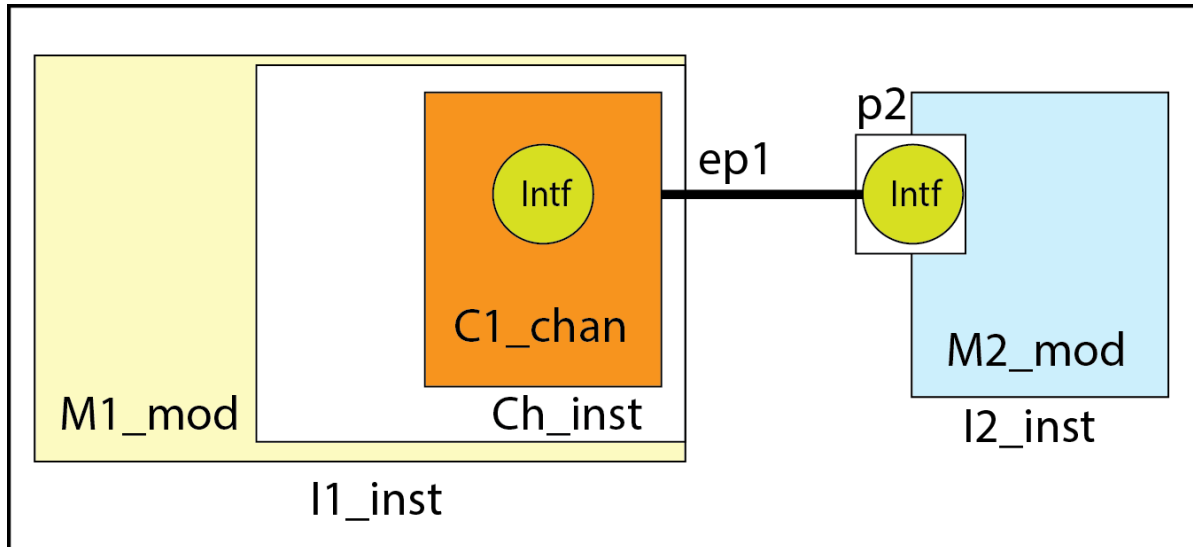
```
SC_MODULE(M1_mod) {
    sc_port<Intf> p1;
    SC_CTOR(M1_mod) {
    }
};

SC_MODULE(M2_mod) {
    sc_port<Intf> p2;
    SC_CTOR(M2_mod) {
    }
};

SC_MODULE(Top) {
    C1_chan Ch_inst;
    M1_mod I1_inst;
    M2_mod I2_inst;
    SC_CTOR(Top) : I1_inst("I1_inst"),
                  I2_inst("I2_inst") {
        I1_inst.p1(Ch_inst);
        I2_inst.p2(Ch_inst);
    }
};
```

Instance Connections: sc_export

- `sc_export<T>` is similar to `sc_port<T>` in declaration/syntax
- Allows moving the channel inside the defining module
 - Hides some of the connectivity details
 - Using the port externally as though it were a channel



```
SC_MODULE(M1_mod) {
    sc_export<C1_chan> ep1;
    C1_chan Ch_inst;
    SC_CTOR(M1_mod) {
        ep1(Ch_inst);
    }
};

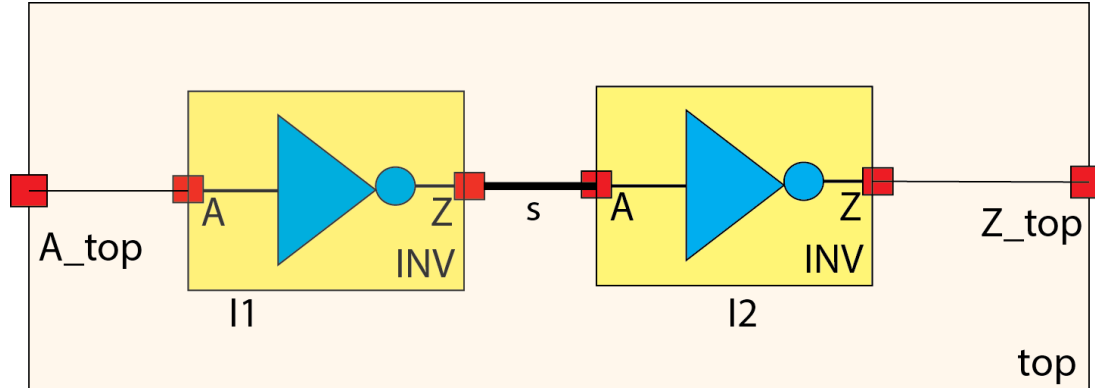
SC_MODULE(M2_mod) {
    sc_port<Intf> p2;
    SC_CTOR(M2_mod) {
    }
};

SC_MODULE(Top) {
    M1_mod I1_inst;
    M2_mod I2_inst;
    SC_CTOR(Top) : I1_inst("I1_inst"),
        I2_inst("I2_inst") {
        I2_inst.p2(I1_inst.ep1);
    }
};
```

Special Ports: `sc_in<T>`, `sc_out<T>`, `sc_inout<T>`

- Predefined ports on interfaces that allow easy manipulation (especially related to events)
 - `sc_in<T>`: specialized port to be used with signals for input (defined on `sc_signal_in_if` interface): data members are `read()`, `event()` [returns true/false], `value_changed_event()`
 - `sc_inout<T>`: specialized port to be used with signals for output (defined on `sc_signal_out_if` interface): data members `write()`, and others of `sc_in<T>`
 - `sc_out<T>`: derived from `sc_inout`
- Further specialization for `bool` and `sc_logic`
- `sc_in<bool>` or `sc_in<sc_logic>` or `sc_inout<bool>` or `sc_inout<sc_logic>`
 - `posedge_event()`, `neg_edge_event()` [returns event]
 - `pos_edge()`, `neg_edge()` [returns true/false]

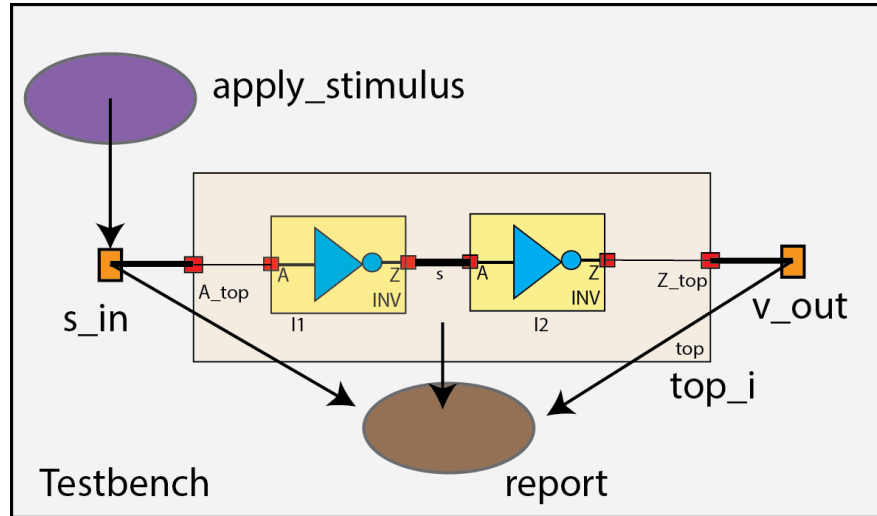
Making Connections: Instances



```
SC_MODULE(INV) {  
    void compute(void); // process  
    sc_in<bool> A;  
    sc_out<bool> Z;  
    SC_CTOR(INV) {  
        SC_THREAD(compute); // register process  
    }  
};  
void INV::compute(void) {  
    while (1) {  
        Z->write(!A->read()); // write to the port  
        wait(A.value_changed_event());  
    }  
}
```

```
SC_MODULE(top) {  
    sc_in<bool> A_top;  
    sc_out<bool> Z_top;  
    INV I1;  
    INV I2;  
    sc_signal<bool> s;  
    SC_CTOR(top) : I1("I1"), I2("I2") {  
        I1.A(A_top); // direct connection  
        I1.Z(s); // connection using channel  
        I2.A(s); // connection using channel  
        I2.Z(Z_top); // direct connection  
    }  
};
```

Testbench

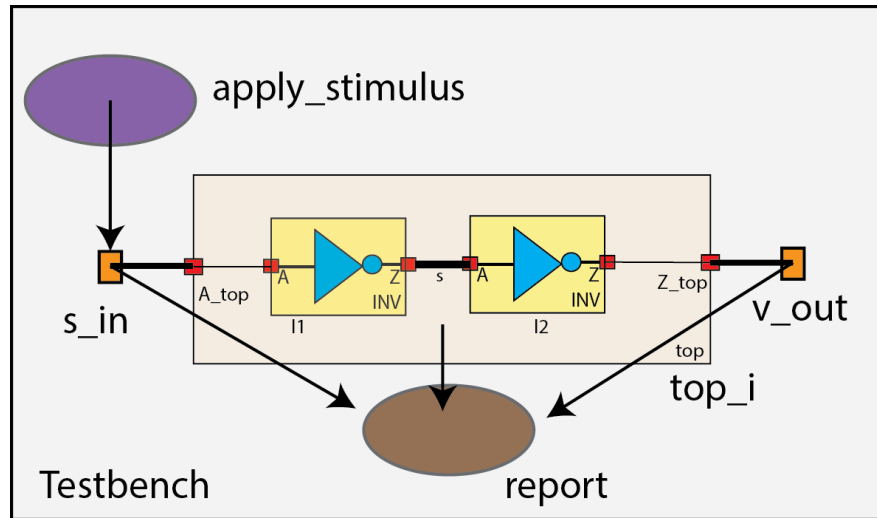


```
SC_MODULE(Testbench) {
    void apply_stimulus(void); // process
    void report(void); // process
    top top_i;
    sc_signal<bool> s_in;
    sc_signal<bool> v_out;
    SC_CTOR(Testbench) : top_i("top_i") {
        top_i.A_top(s_in);
        top_i.Z_top(v_out);
        SC_THREAD(apply_stimulus);
        SC_THREAD(report);
    }
};
```

```
void Testbench::apply_stimulus(void)
{
    wait(SC_ZERO_TIME);
    s_in.write(true); // apply stimulus
    wait(1, SC_SEC);
    wait(SC_ZERO_TIME);
    s_in.write(false); // apply stimulus
}
```

```
void Testbench::report(void)
{
    while (1) {
        cout << "Input=" << s_in.read()
              << " top/A:" << top_i.A_top->read()
              << " top/I1/A:" << top_i.I1.A->read()
              << " top/I1/Z:" << top_i.I1.Z->read()
              << " top/I2/A:" << top_i.I2.A->read()
              << " top/I2/Z:" << top_i.I2.Z->read()
              << " top/Z:" << top_i.Z_top->read()
              << " Output:" << v_out.read()
              << " at: " << sc_time_stamp() << endl;
        wait(s_in.value_changed_event() |
             top_i.A_top.value_changed_event() |
             top_i.I1.A.value_changed_event() |
             top_i.I1.Z.value_changed_event() |
             top_i.I2.A.value_changed_event() |
             top_i.I2.Z.value_changed_event() |
             top_i.Z_top->value_changed_event() |
             v_out.value_changed_event());
    }
}
```

Testbench: Output



- After multiple delta cycles, stable values obtained
- Can use the trace file to create VCD file and see the results in the waveform viewer

```
Input=0 top/A:0 top/I1/A:0 top/I1/Z:0 top/I2/A:0 top/I2/Z:0 top/Z:0 Output:0 at: 0 s
Input=0 top/A:0 top/I1/A:0 top/I1/Z:1 top/I2/A:1 top/I2/Z:1 top/Z:1 Output:1 at: 0 s
Input=1 top/A:1 top/I1/A:1 top/I1/Z:1 top/I2/A:1 top/I2/Z:0 top/Z:0 Output:0 at: 0 s
Input=1 top/A:1 top/I1/A:1 top/I1/Z:0 top/I2/A:0 top/I2/Z:0 top/Z:0 Output:0 at: 0 s
Input=1 top/A:1 top/I1/A:1 top/I1/Z:0 top/I2/A:0 top/I2/Z:1 top/Z:1 Output:1 at: 0 s
Input=0 top/A:0 top/I1/A:0 top/I1/Z:0 top/I2/A:0 top/I2/Z:1 top/Z:1 Output:1 at: 1 s
Input=0 top/A:0 top/I1/A:0 top/I1/Z:1 top/I2/A:1 top/I2/Z:1 top/Z:1 Output:1 at: 1 s
Input=0 top/A:0 top/I1/A:0 top/I1/Z:1 top/I2/A:1 top/I2/Z:0 top/Z:0 Output:0 at: 1 s
```

References

- Black, David C., and Jack Donovan, eds. SystemC: From the ground up. Boston, MA: Springer US, 2004.